

A Federated Knowledge Graph Architecture for Dutch Cadastral Data

Stefan Bussemaker^{1,2} , Sven Mol^{1,2} , Lei Wang^{1,2} , João Rebelo Moreira² ,
and Alexandra Rowland^{1,2} 

¹ Kadaster, Hofstraat 110, 7311 KZ Apeldoorn, The Netherlands

² University of Twente, Drienerlolaan 5, 7522NB Enschede, The Netherlands

Abstract. Kadaster, the Dutch National Land Registry and Mapping Agency, has been actively publishing their base registries as linked (open) spatial data for several years. To date, a number of these base registers as well as a number of external datasets, have been successfully published as linked data and are publicly available in the form of the Kadaster Knowledge Graph (KKG). While the KKG provides a unified query interface across heterogeneous datasets, its centralised batch architecture produces stale data, imposes fragile pipeline maintenance, and requires copying third-party data into Kadaster’s infrastructure, creating duplication and governance overhead. This paper proposes a federated architecture in which Kadaster and its partner organisations each expose their data through a Model Context Protocol (MCP) server, and a centralised AI assistant orchestrates queries across these endpoints at runtime using tool calling. The proposed architecture eliminates the extract, transform, load (ETL) pipeline and triple store, keeps data current at its authoritative source, and enables natural language querying over the combined data landscape. We designed the architecture using ArchiMate and validated it through a proof-of-concept implementation. We discuss the limitations of this approach and discuss possible remedies.

Keywords: Knowledge Graph · Federated Architecture · Model Context Protocol · Virtual Knowledge Graph · Ontology Matching

1 Introduction

The Dutch Cadastre, Land Registry and Mapping Agency (Kadaster) is the authoritative source of administrative and spatial data on property and ownership rights in the Netherlands, maintaining several national key registers including the Key Register for Addresses and Buildings (BAG), Key Register for Topography (BRT), the Key Register for Large Scale Topography (BGT), and the Cadastral Key Register (BRK). These registers underpin Kadaster’s core mission of guaranteeing legal certainty (*rechtszekerheid*), but are increasingly called upon to support broader societal purposes: answering cross-register geospatial questions, supporting regional planning, and combining Kadaster data with that of partner organisations.

Effective data sharing across this ecosystem is non-trivial. The key registers are independently managed by distinct institutions, a structural reality which both necessitates and motivates a decentralised operational model, modelled for different purposes, and governed under distinct institutional constraints. In practice, this results in a system of data silos. Over the past decade, Kadaster has addressed this through a series of linked data initiatives culminating in the Kadaster Knowledge Graph (KKG): a materialised knowledge graph built using an extract, transform, load (ETL) pipeline, which is run every three months. The resulting triples are stored and served through TriplyDB, which is a triple store product often used for KGs [7,6]. While the KKG provides a unified view across registers and external datasets, its centralised, batch-oriented architecture has well-documented limitations: data is routinely stale; the ETL pipeline is fragile and expensive to maintain; and data from partner organisations must be copied into Kadaster’s infrastructure, creating duplication and governance overhead.

Virtual Knowledge Graphs (VKGs) offer a principled alternative, keeping data at source and rewriting queries at runtime against a shared ontology [13]. This paper proposes an ecosystem in which Kadaster and adjacent organisations each expose their data through a Model Context Protocol (MCP) server [1], enabling the AI orchestrator Loki³ to access distributed sources in a structured way.

The proposed architecture is based on a lightweight, request-driven integration model that eliminates the ETL pipeline and triple store, and enables natural language querying over the combined data landscape. A proof-of-concept (POC) was performed through the development of a prototype that illustrates the application of the architecture for the KKG use case with synthetic data. The results showed the need for an ontology matching approach for the integration of the involved ontologies.

The remainder of this paper is structured as follows. Section 2 describes the background. Section 3 presents the current baseline architecture. Section 4 presents the proposed federated architecture. Section 5 validates the approach through a proof-of-concept implementation. Section 6 discusses the multi-ontology challenge. Section 7 concludes.

2 Background

2.1 Semantic Web Technologies

The Semantic Web provides a framework for representing and querying structured knowledge using standardised, machine-readable technologies [11]. Data is represented in the form of Resource Description Framework (RDF) triples, enabling interoperable data exchange across heterogeneous systems. SPARQL is the standard query language for RDF datasets and is currently the primary interface through which the Kadaster Knowledge Graph is queried. Ontologies,

³ Loki documentation

expressed in languages such as the Web Ontology Language (OWL) [10], define shared conceptualisations of a domain and support semantic interoperability across sources. Supporting standards used in this work include the Shapes Constraint Language (SHACL) [12] for validating RDF data and the Simple Knowledge Organisation System (SKOS) [9] for aligning concept vocabularies across organisations.

2.2 Knowledge Graphs and Virtual Knowledge Graphs

A knowledge graph (KG) integrates heterogeneous data into a unified graph structure enriched with ontological semantics [4]. Knowledge graphs are increasingly used in enterprise and government settings to support cross-domain querying and data integration. A *materialised* knowledge graph pre-computes and persists the integrated graph, enabling efficient query processing at the cost of data freshness and pipeline overhead.

A VKG maps ontology terms to source schemas and rewrites queries into native source queries at runtime [13], eliminating the need for ETL pipelines and ensuring that results always reflect the current state of the source data. The Ontop system is a widely used implementation of this approach for relational databases [13].

2.3 Model Context Protocol

MCP is an open protocol that standardises how large language models (LLMs) connect to external tools and data sources [1]. MCP defines a client-server architecture in which an LLM host connects to MCP servers that expose capabilities as tools, resources, and prompts. Tools are invoked via JSON Remote Procedure Call (JSON-RPC) calls. The `tools/list` procedure allows a client to discover available tools and their schemas at runtime, while `tools/call` executes a specific procedure. This enables LLM-based agents to dynamically invoke external services at inference time, supporting request-driven integration without pre-materialised data. In the context of this work, MCP serves as the integration layer through which the AI assistant Loki discovers and accesses distributed data sources at query time, acting as both an orchestrator and a service registry for endpoint routing.

2.4 Ontology Matching

When data sources use heterogeneous ontologies, a shared semantic foundation must be established before integration is possible. Ontology matching is the process of identifying correspondences between concepts, properties, and instances in different ontologies [2]. These correspondences can be expressed using alignment languages, which supports explicit mappings between heterogeneous ontologies where direct alignment is not possible. In this work, ontology matching arises as a key challenge when federating over partner organisations that model

their data differently, and is addressed through a combination of graph-based structural similarity, semantic embedding techniques, and LLM-assisted mapping generation.

3 Baseline Architecture

3.1 Multi-Party Ecosystem

Kadaster operates within a complex organisational structure. For brevity, only the parts relevant to the current case are considered. Internally, Kadaster consists of several departments responsible for managing data pipelines and knowledge graph infrastructure, maintaining geospatial datasets, and creating semantic models for data sharing purposes in the public sector. The key registers are independently managed by distinct institutions. Among the data suppliers are governmental bodies whose data is used to update key registers. Datasets with geographic features are created and supplied for publication on national map services. Data consumers include public organisations as well as citizens or companies who may have geographic questions that can only be answered by combining datasets from Kadaster and other data sources.

The main driver for Kadaster is accurate property registration, requiring data to be trusted to guarantee legal certainty. Kadaster is also involved in solving societal challenges with geospatial data, combining datasets from multiple registries across the public sector. Regional authorities use this data for analysis to improve services for their citizens. Accessibility for users is also important: such administrative and geospatial data are difficult for non-technical users to access directly, for example when interaction requires SPARQL knowledge. An AI assistant such as Loki can reduce this barrier by allowing users to express their information needs in natural language, while the underlying system interprets and translates these requests into structured data access.

3.2 Application Architecture

Figure 1 provides an overview of the information structure of the baseline centralised KKG architecture. It shows how data from external and internal sources is materialised in the centralised graph through a pipeline consisting of data ingestion, ETL transformation, RDF triple generation, and triplestore persistence [4]. The AI assistant Loki acts as the user-facing interaction layer, responding based on the data already loaded into the KKG.

The baseline application architecture operates within an ecosystem consisting of external, partner, and internal components. External sources expose foundational datasets through dedicated services, while internal components interact through a processed data service, transformation service, RDF triple store service [11], KKG query service, and response service. The data retrieval and answer generation process starts with a user request, which triggers access to external data services. The retrieved datasets are ingested and processed, transformed

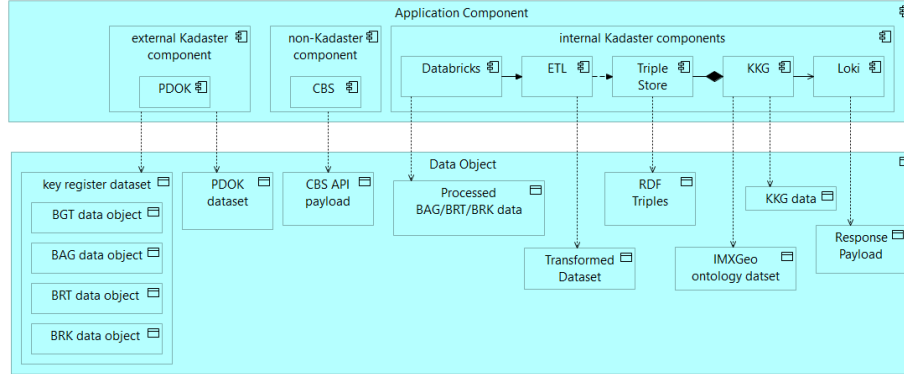


Fig. 1. Information Structure baseline: the structure of key datasets (BAG, BGT, Statistics Netherlands (CBS)) as materialised in the Kadaster Knowledge Graph.

via ETL and RDF services, and persisted in the triple store. Subsequently, the KKG query service and Loki response service are invoked to generate a response to the user.

3.3 Implementation and Deployment Architecture

Key register data is supplied to Kadaster by the authoritative source holders, who are responsible for maintaining it within their own domains. Kadaster processes and stores the received data in separate dedicated databases. For creating the KKG, the ETL pipeline runs in the public cloud on Azure Databricks, where the data is processed and harmonised, and triples are created. These triples are then uploaded to the triple store database, where the KKG is hosted. Each team maintains independent infrastructure and applications.

The key registers are processed and stored in Kadaster’s private cloud environment. Applications run on dedicated servers connected to their own databases. A connection exists to the public cloud environment so that data from the key registers can be processed there. The resulting triples are stored in TriplyDB, through which the KKG is exposed. Partner organisations run their own infrastructure and expose their data through publicly accessible HTTP services. The full deployment is shown in Figure 2. This centralised materialisation approach introduces latency and governance overhead when source registries update independently [5,3], motivating the federated architecture proposed in Section 4.

3.4 Problem Analysis

Although the baseline architecture provides an integrated view across multiple datasets, it also introduces several limitations that affect system maintenance and operation. A major limitation lies in the periodicity of the ETL process.

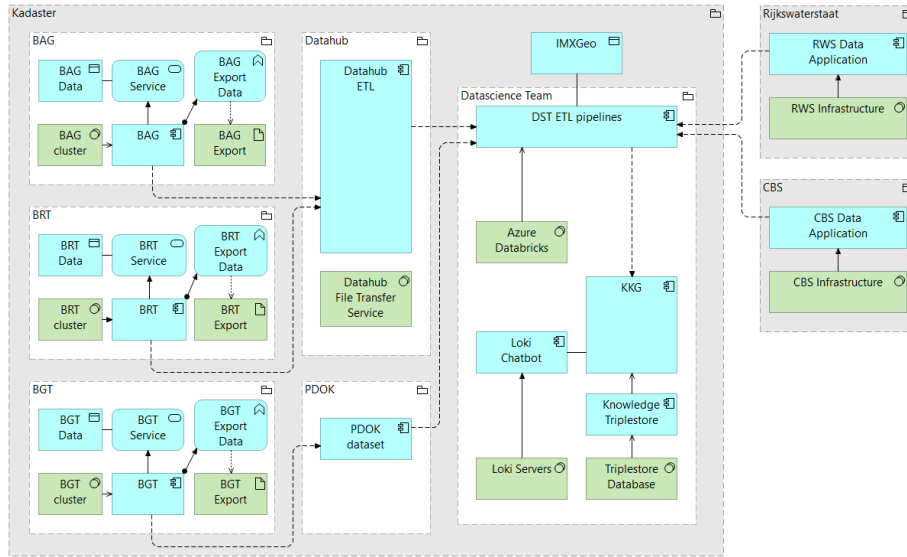


Fig. 2. Implementation and Deployment baseline: the physical infrastructure spanning Kadaster’s private cloud, public cloud, and externally hosted services.

Since the KKG is updated on a quarterly cycle and the data loading process itself can take roughly a week, the architecture is ill-suited for the timely incorporation of changes from source registries. This creates systematic delays between the source updates and their availability in the integrated graph, limiting the system’s ability to support time-sensitive or frequently changing information needs. A related concern is pipeline fragility. Changes in source schema, data quality, or transformation rules can disrupt the ETL process or require manual intervention. Maintaining the system, therefore, demands continuous monitoring and technical effort, particularly given the heterogeneity of the connected data sources. This fragility compounds as the number of sources grows, with each additional source introducing new schema assumptions, transformation rules, and potential failure points.

Data freshness is a direct consequence of the batch-oriented update model. Because the integrated graph reflects the state of its sources only at the time of the last ETL run, users may access information that is months out of date. This affects not only the accuracy of query results but also user trust in answers generated by applications built on top of the graph. Finally, the ETL process requires copying data from external organisations into Kadaster’s infrastructure, causing data duplication and governance overhead. Responsibilities for storage, synchronisation, and provenance tracking are drawn into Kadaster’s remit even for data it does not originate. Together, these four limitations show that the centralised, batch-oriented architecture introduces operational rigidity and gov-

ernance challenges that become increasingly acute as the system scales across more dynamic and distributed data sources.

4 Target Architecture: Virtual Knowledge Graph via MCP-Based Federation

4.1 Realisation of Goals and Requirements

This section presents the proposed target architecture and explains how it directly addresses the four limitations identified in Section 3. The architecture migrates the KKG to a VKG [13] in which Kadaster and partner organisations each expose their data through MCP servers, and Loki orchestrates queries across these endpoints at runtime. In this paper, we use the term MCP-KKG to refer to this VKG variant, where MCP servers and tool calling provide the federation mechanism rather than relying primarily on SPARQL rewriting. The remainder of this section details the application and technology changes required to realise this approach.

4.2 Application Layer

Compared with the baseline, the target architecture presents a request-driven and ontology-aligned integration model with reduced component complexity and operational overhead. The core internal components are the AI assistant (Loki), the MCP server, and the VKG. External data sources are accessed through the MCP tool service, decoupling data providers from internal orchestration logic. Loki relies on a decision-making service to discover and select relevant tool invocations [8], and generates responses via the response service. The decision-making role is important because the federating proxy must route user requests to the relevant data source. As the number of connected parties and tools increases, maintaining manual routing rules becomes increasingly difficult. The MCP server exposes the MCP tool to mediate access to external sources, while the VKG is accessed through the VKG service. Compared to the baseline, Databricks, the ETL pipeline, and the triple store are removed entirely.

A user request is interpreted by Loki, which discovers and selects the relevant tool via the decision-making service. The MCP server then invokes the appropriate tool on demand to retrieve ontology-aligned data directly from the source, and Loki assembles the result into a response payload.

The Information Structure Viewpoint, shown in Figure 3, presents the key data objects exchanged between the application components. Loki governs tool invocation through the decision-calling tool, response payload, and MCP tool service. The MCP server is characterised by shared ontology datasets and MCP tool schemas, which establish the semantic foundation for interpreting and integrating data from multiple sources. The VKG schema clarifies the conceptual structure of the virtual graph, enabling data from heterogeneous sources to be understood uniformly.

4.3 Technology Layer

In the proposed architecture, the transformation pipeline within Kadaster is no longer needed. The architecture assumes that a shared ontology is already in use across all participating organisations, and that each organisation exposes data already aligned to this ontology through their MCP server. As a result, no mapping or harmonisation step is required at query time, as data retrieved from any source can be interpreted uniformly without transformation.

On the technology layer, each organisation or key register exposes a JSON-RPC server accessible externally. The servers can run on any infrastructure chosen by the responsible teams, reducing Kadaster’s infrastructure footprint. From Kadaster’s perspective, direct interfaces with data source providers replace the internal data platform and its associated processing pipelines. This frees teams to focus on developing and maintaining the natural language querying capability. The overall technological architecture is shown in Figure 4.

4.4 Comparison with the Baseline Architecture

Table 1 summarises the key architectural differences between the baseline and target architectures across four dimensions relevant to enterprise architecture evaluation. The comparison shows that the target architecture addresses several limitations of the baseline architecture, but also redistributes some responsibilities and risks to other parts of the system.

The target architecture improves data freshness by accessing data directly from the source at query time, rather than relying on periodic ETL updates to the central graph. This reduces delays between source updates and their availability in the integrated graph, making the system more suitable for frequently changing information needs. However, it also introduces a dependency on external endpoints. If an endpoint is unavailable, slow, or fails, query results may be delayed or incomplete.

On the question of semantic integration, the baseline architecture handles harmonisation during ETL before loading data into the KKG, which increases

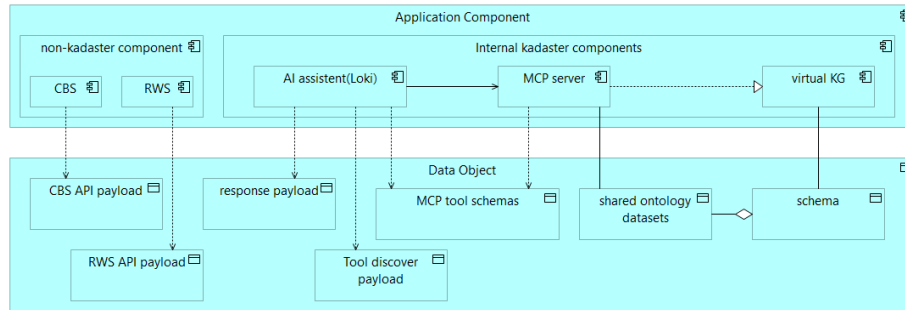


Fig. 3. Information Structure target: data objects governing tool invocation, ontology alignment, and VKG schema across application components.

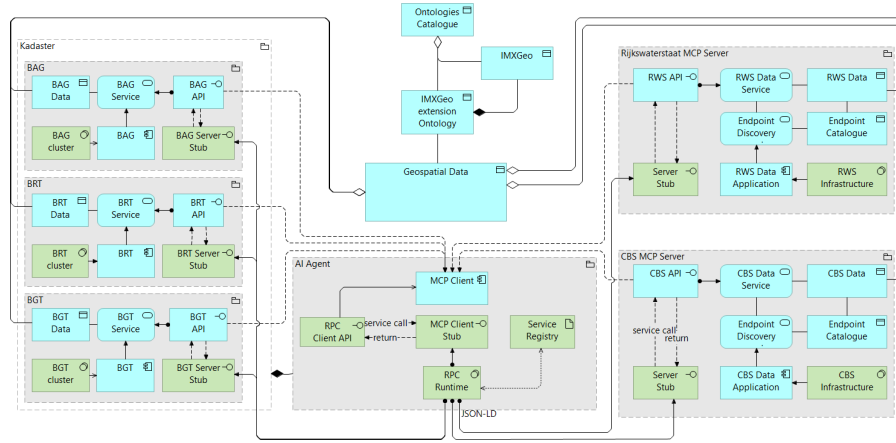


Fig. 4. Implementation and Deployment target: Kadaster and partner organisations each expose an MCP server; data is exchanged as JSON for Linked Data (JSON-LD) over JSON-RPC following a common ontology.

Table 1. Comparison of baseline and target architectures.

Dimension	baseline (KKG)	target (VKG)
Data freshness	Batch update every 3 months	Real-time at query time
Semantic integration	Harmonised during ETL	Shared ontology agreement
Data ownership	Centralised at Kadaster	Distributed at source organisations
Governance responsibility	Kadaster controls full pipeline	Shared across organisations

maintenance effort whenever source schemas or transformation rules change. In the target architecture, semantic integration rests on a shared ontology agreement, allowing distributed sources to be accessed through MCP servers and VKG services without repeated ETL harmonisation. However, this shifts complexity to coordination, requiring all organisations to align on a shared ontology before sources can be queried consistently at runtime.

Regarding data ownership, the baseline architecture centralises copied data within Kadaster, granting it both control over the data and full responsibility for storage, synchronisation, and provenance. In the target architecture, source organisations retain control over their own data, reducing duplication and clarifying ownership. Governance responsibility becomes shared, however, requiring ongoing coordination of ontology alignment, update policy, and endpoint availability.

Overall, the target architecture improves data freshness, reduces data duplication, and lowers central maintenance overhead, but redistributes complexity towards runtime orchestration, shared governance, and semantic integration co-

ordination. The traceability and governance implications of this redistribution are discussed further in Section 6.

5 Validation

To validate the proposed architecture, a POC was implemented that demonstrates the technical working of its main architectural features.⁴ The scope of the POC is the data exchange between services by showing the interfaces, data formats, and semantics. This shows how services can be developed and maintained in isolation, while still allowing data from different sources to be combined dynamically. The objective is to provide a standardised interface between the key registers in Kadaster and the external partner data sources. Rather than every service having its own API design, the services offer the same endpoints, following the MCP protocol as described in this section. Data is transported by `stdio`.

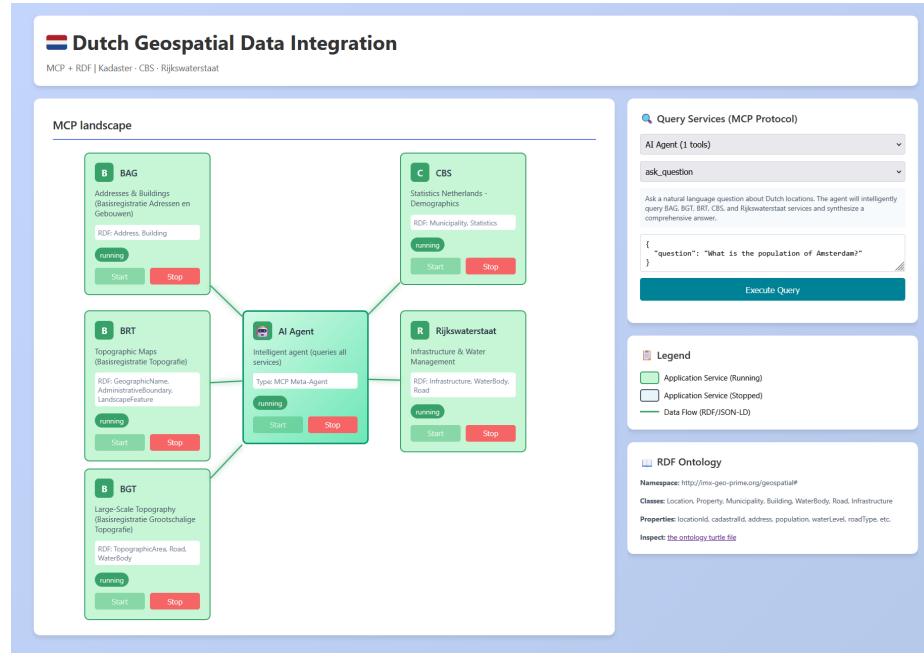


Fig. 5. POC dashboard: the MCP landscape exposes five servers across Kadaster and its partner organisations. The AI Agent service accepts natural language questions and returns answers grounded in the shared RDF ontology.

⁴ Source code available at <https://github.com/sbussemaker/FKG-Architecture-for-Dutch-Cadastral-Data>.

Figure 5 shows the POC dashboard. Each of the five MCP servers exposes a data source: three cover Kadaster’s base registries and two cover partner organisations. The AI Agent in the centre is the single entry point through which users pose questions in natural language.

5.1 Tools

A tool in this context means a JSON-RPC call with additional agreements. The POC uses three MCP procedures: `initialize`, `tools/list`, and `tools/call`, with identical stubs across all servers. Because `tools/list` includes each procedure’s documentation and input schema alongside its name, calls carry a larger payload than conventional API calls. For example, the BAG server exposes four tools: `find_address`, `list_addresses`, `get_address`, and `get_building`. Each entry includes a name and a natural-language description with explicit guidance (such as prerequisites and return types) that steers the LLM’s tool selection.

```
{
  "method": "tools/call",
  "params": {
    "name": "get_building",
    "arguments": {
      "location_id": "LOC001"
    },
    "_meta": {
      "progressToken": 1
    }
  }
}
```

(a) MCP tool call request

```
{
  "@context": {
    "geo": "http://imx-geo-prime.org/
      geospatial#"
  },
  "@id": "http://imx-geo-prime.org/bag/
    buildings/BAG-036301001234",
  "@type": "geo:Building",
  "geo:locationId": "LOC001",
  "geo:buildingPurpose": "office",
  "geo:constructionYear": "1920",
  ...
}
```

[h] (b) JSON-LD semantic response

Fig. 6. MCP `tools/call` request and its semantically enriched JSON-LD response from the base registry server.

Once the client has discovered available procedures, a decision must be made on which ones to invoke. There are multiple strategies for this, discussed in Section 5.3. Figure 6 above shows the request and response for a sample call to the BAG MCP server querying building information. The `tools/call` procedure is invoked with the function name and arguments as parameters. The BAG `get_building()` function returns the data it has available about the location in linked data format; in principle any data can be returned, as long as it conforms to the JSON-LD format.

5.2 Semantics

While MCP provides a convenient protocol for exchanging data in a standardised way, it lacks built-in mechanisms for data semantics, which are necessary for full interoperability. MCP does offer an optional `outputSchema` description

in the `tools/list` procedure, but this only conveys the data format. Knowing that a response field is an integer is not sufficient for semantic interoperability. The shared ontology introduced in Section 4 serves this purpose. In the POC, every data provider exposes its data already aligned to the shared `http://imx-geo-prime.org/geospatial` ontology, as illustrated in the JSON-LD response in Figure 6. The ontology URI and data values used throughout this POC are illustrative; a production deployment would align to an established domain standard such as IMX-Geo or NEN 3610.

5.3 AI Agent

With standardised MCP server endpoints and a shared ontology in place, the POC demonstrates an AI Agent implementation that integrates data across sources. The AI Agent is configured with all MCP servers it may query and builds a list of client stubs by calling all `tools/list` procedures at startup. The tool schemas collected at startup are passed to GPT-4o mini via its function-calling API; the model selects which tool(s) to invoke and with what arguments based on the user’s natural language question. The agent executes the corresponding JSON-RPC calls, receives the JSON-LD results, and constructs a final response using the same model. Because all responses share the same format and ontology, data from different sources can be integrated without additional transformation. This directly realises the Loki orchestration role described in Section 4.

To illustrate, consider a query asking for a complete profile of Amsterdam, covering building types, demographics, infrastructure status, terrain, parks, and administrative authorities. This demonstrates the architecture’s core capability: a single natural language request transparently spanning five heterogeneous sources without manual join logic. The agent automatically selected and invoked tools across all five MCP servers, one per source: `get_building` (BAG), `get_statistics` (CBS), `get_infrastructure` (Rijkswaterstaat), `get_terrain` (BGT), and `get_landscape` and `get_boundaries` (BRT). The merged response covered building characteristics (construction year: 1920, surface area: 4,500 m²), demographics (population: 872,680, average income: EUR 38,500 from CBS), infrastructure status (A10 highway condition from Rijkswaterstaat), and administrative boundaries (municipality, province, water board from BRT), all in a single coherent answer. No manual join logic was required. The shared ontology and uniform JSON-LD response format allowed the agent to transparently combine results from five heterogeneous sources. This directly addresses the shortcomings identified in Section 3: data is accessed at query time rather than copied in batch, eliminating both the quarterly staleness and the governance overhead of hosting third-party data at Kadaster.

Several aspects fall outside the scope of this POC. The POC excludes legally restricted data sources; extending the architecture to closed data requiring authorisation is left for future work. Additionally, all MCP servers communicate over `stdio` and therefore run on a single machine; a production deployment would require network-accessible endpoints. The data and ontology URIs used are illustrative rather than drawn from live registries. The POC covers five data

sources; routing and latency behaviour at the scale of Kadaster’s approximately 400 partner organisations remains untested. Finally, the shared ontology is assumed to be pre-agreed; the governance process required to establish and maintain such an ontology in practice is addressed in Section 6.

6 Discussion

6.1 Limitations of a single ontology

While the proposed architecture demonstrates clear advantages over the centralised ETL-based approach, concerns remain regarding its application at scale. Kadaster works with close to 400 partners providing largely similar data, mainly differing in the geographical area they are concerned with. The overlap of these geographical domains can prove challenging to the MCP server, as it will have to decide which API is best suited, even when the question does not explicitly contain the required information to make such a decision. For example, the oldest embankment in a given region may be found in the municipal data or that of the water board, but it may also live in the data of *ProRail* in the case of a railway embankment. It should be obvious that this problem gets exacerbated as the number of integrated data sources increases. A second challenge of the approach is the requirement to establish a single, shared ontology which all parties have to abide by. While such an artefact makes it easier to integrate various systems, its development requires a large amount of reconciliation between the data models currently present in the systems of various data owners. This can prove to be too costly or consume too much time to be feasible. Furthermore, the number of involved parties makes it hard to change this ontology at a later date. Even the smallest changes in the ontology will require all parties to update their systems to maintain compatibility with the central MCP server. While various architectures exist to ease such a transition, it needs to be applied to many hundreds of systems.

6.2 Motivation for a multi-ontology approach

Another approach in the early stages of investigation is a multi-ontology approach, which addresses the limitations by accepting the inherent heterogeneity of data. This approach defines a canonical ontology and maps all connected data sources to it. This allows data owners to present the data in a shape most applicable to their situation. The envisioned system then maps questions to the respective ontology, and maps responses back to the canonical ontology. This enables a looser coupling between the proxy and source systems.

The process of aligning the canonical ontology with source-specific ontologies is achieved through a combination of semantic techniques and graph-based approaches. Graph-based approaches analyse the structural similarity of different ontologies to find correspondences between them, for example, using topological patterns and similarity measures of individual nodes. Semantic techniques are

used to identify and resolve both ambiguities and differences in terminology, for example, using embedding models.

To support the creation of the required ontology mappings, LLM-based systems can be leveraged to aid human integrators. Less advanced techniques, such as graph and textual similarity, can be used as a first step to aid the LLM system. The results of the LLM can be validated using graph-based and rule-based checks. When the canonical ontology of the system requires updating, or when a data source system is changed, the system can be reconfigured easily to adapt to the change as a result of the loose coupling. This has the added benefit that changes can also be tested and rolled out in a phased approach, should the need for that arise. Table 2 summarises the key architectural differences between the shared and multi-ontology approaches.

Table 2. Comparison of shared and multi-ontology approaches

Dimension	Shared ontology	Multi-ontology
Data access	Through MCP tools	Through generic APIs
Integration mechanism	Shared ontology and MCP tool schemas	Canonical ontology and mappings
Ontology usage	Single shared ontology	Multiple local ontologies
Infrastructure impact	Reduces central infrastructure but requires MCP servers	Reduces coupling but requires mapping management
Main limitation	Shared ontology requirement Routing complexity Traceability issues	Mapping quality Runtime complexity Governance of mappings

6.3 Relation with Broader Data Architecture Trends

The limitations identified in the baseline architecture, data staleness, pipeline fragility, data duplication and centralised governance are all well-documented characteristics of centralised data architectures more generally, including data warehouses, data lakes and their hybrid successors. The Data Mesh paradigm represents an influential response to these limitations, advocating for domain-oriented, decentralised data ownership in which each domain exposes its data as a product through standardised interfaces; a philosophy closely aligned with the federated architecture proposed in this work. Where Data Mesh provides an organisational and governance framework, the architecture proposed here offers a concrete technical realisation of its principles in the context of Dutch administrative data, with MCP servers serving as the standardised interface layer, source organisations retaining ownership of their data, and Loki providing a domain-agnostic query entry point. The multi-ontology approach discussed above further echoes Data Mesh’s acceptance of domain-local data models, addressing interoperability through mapping rather than enforced convergence.

7 Conclusion

The Kadaster Knowledge Graph provides an integrated view across multiple data sources, but at the cost of data freshness: its centralised, batch-oriented architecture produces quarterly snapshots that may be months out of date by the time they are queried. Alternative real-time approaches exist. VKGs such as On-top offer query-time access but scale poorly to large and heterogeneous datasets; semantic reasoning requires data to be held locally; and federated SPARQL constrains all participants to a single query interface; none of which fully addresses the operational and governance challenges identified in this paper.

The proposed MCP-based federated architecture addresses these limitations by providing a standardised way for data sources to expose their APIs, solving the technical challenge of endpoint heterogeneity without requiring centralised data copying. Combined with the ontology matching techniques discussed in Section 6.2, semantic harmonisation can be performed at the application layer and at query time, rather than as a costly initial ETL step. The proof-of-concept demonstrates that a single natural language query can transparently span five heterogeneous sources without manual join logic, directly realising the Loki orchestration vision.

Two open challenges remain. The first is performance: fetching data over the network at query time introduces latency for each request, compounded when multiple data providers must be consulted. The second is quality: because integration occurs at query time, results may be incomplete if a source is unavailable or slow, and systematic quality checks, straightforward in a batch ETL pipeline, become the responsibility of the consuming application. Addressing these challenges, along with extending the architecture to authorisation-restricted data sources and validating routing behaviour at the scale of Kadaster’s approximately 400 partner organisations, constitute the primary directions for future work.

References

1. Anthropic: Model context protocol specification. Tech. rep., Anthropic (2024), <https://modelcontextprotocol.io/specification/>
2. Euzenat, J., Shvaiko, P.: Ontology matching. Springer (2007)
3. Halevy, A., Rajaraman, A., Ordille, J.: Data integration: The teenage years. In: Proceedings of the 32nd international conference on Very large data bases. pp. 9–16 (2006)
4. Hogan, A., Blomqvist, E., Cochez, M., d’Amato, C., Melo, G.D., Gutierrez, C., Kirrane, S., Gayo, J.E.L., Navigli, R., Neumaier, S., et al.: Knowledge graphs. *ACM Computing Surveys (Csur)* **54**(4), 1–37 (2021)
5. Lenzerini, M.: Data integration: A theoretical perspective. In: Proceedings of the twenty-first ACM SIGMOD-SIGACT-SIGART symposium on Principles of database systems. pp. 233–246 (2002)
6. Rowland, A., Folmer, E., Beek, W., Schevers, H., Kolk, A.: Lessons learned in the pursuit of production-readiness: Iterative evaluation of the kadaster knowledge graph construction (2024), submitted to Semantic Web Journal

7. Rowland, A., Folmer, E., Beek, W., Wenneker, R.: Interoperability and integration: An updated approach to linked data publication at the dutch land registry. *ISPRS International Journal of Geo-Information* **11**(1), 51 (2022). <https://doi.org/10.3390/ijgi11010051>
8. Schick, T., Dwivedi-Yu, J., Dessì, R., Raileanu, R., Lomeli, M., Hambro, E., Zettlemoyer, L., Cancedda, N., Scialom, T.: Toolformer: Language models can teach themselves to use tools. *Advances in neural information processing systems* **36**, 68539–68551 (2023)
9. W3C: SKOS simple knowledge organization system reference. Tech. rep., World Wide Web Consortium (2009), <https://www.w3.org/TR/skos-reference/>
10. W3C: OWL 2 web ontology language primer (second edition). Tech. rep., World Wide Web Consortium (2012), <https://www.w3.org/TR/owl2-primer/>
11. W3C: RDF 1.1 concepts and abstract syntax. Tech. rep., World Wide Web Consortium (2014), <https://www.w3.org/TR/rdf11-concepts/>
12. W3C: Shapes constraint language (SHACL). Tech. rep., World Wide Web Consortium (2017), <https://www.w3.org/TR/shacl/>
13. Xiao, G., Lanti, D., Kontchakov, R., Komla-Ebri, S., Güzel-Kalaycı, E., Ding, L., Corman, J., Cogrel, B., Calvanese, D., Botoeva, E.: The virtual knowledge graph system ontop. In: *Proceedings of the 19th International Semantic Web Conference (ISWC)*. *Lecture Notes in Computer Science*, vol. 12507, pp. 259–277. Springer (2020). https://doi.org/10.1007/978-3-030-62466-8_17